Kipu Anon · Doc de diseño (Reto 1: Vende los datos sin vender a la gente)

Ismael Florean · Frontier CDMX/26 · Repo: github.com/SamaelFL/kipu-anon · Diagrama: docs/diagrama.png

1. Problema, meta y tesis

Hay que entregar el corpus de Kipu Pay (2.028 mensajes, 26 docs, 6.243 filas) uno a uno, sin que nadie sea reidentificable, útil para entrenar y con cada persona como una sola identidad de punta a punta, incluso cuando las fuentes crecen. Puntúa fuga 45, utilidad 35 y consistencia 20; una base solo con reglas da ~88. **Tesis:** el problema no es encontrar nombres, es **saber quién es quién**. Medido sobre el corpus real: de los 149 usuarios-cliente del chat, 77 se ligan al maestro por correo, 63 por un nombre único y **8 son homónimos** que solo se resuelven con evidencia. Además hay un empleado que se llama igual que un cliente (Luis Santander), apodos ("Pipe", "Teban"), apellidos que son lugares ("Santander", "Castilla", "Restrepo", "Bolívar"), terceros fuera de las tablas ("mi hermano Daniel" con su llave, "mi mamá, Ignacio", "mi jefe Kevin Díaz"), y nombres de figuras públicas o marcas que parecen personas (García Márquez, Egan Bernal, Jorge Isaacs, Juan Valdez). Por eso el centro del diseño es un **grafo de identificadores**: los nombres son etiquetas, nunca llaves. Encima van reglas deterministas, un LLM de recall cacheado y un juez local que replica al del reto. El texto se reemplaza, nunca se reescribe.

2. Pipeline (Python con librería estándar; mismo código para `/run` y `/anonymize`)

1. **Ingesta exhaustiva:** API con ETag, backoff y caché. Se leen todos los rincones donde vimos datos personales en el corpus: Info de PDF en UTF-16 (Author, Subject, **Keywords** con nombres), comentarios HTML ("revisado por … pagudelo@gmail.com") y comentarios XML, front matter con `@handles` y correos de terceros, `<meta author>`, nombres de adjuntos, y URLs con slugs o parámetros (`?u=khenao`, `/comercios/gabriela-londono`). `clientes.csv` va en Latin-1. Salida 1:1 con `record_index`.
2. **Grafo de identificadores:** los nodos son llaves fuertes (CL-, KP-, uid, radicado PQR, cédula/CE, correo, celular, placa, pasaporte, n.º de crédito `4471-xxxx`) y las aristas salen de las tablas, de `users.json` (correo → cliente) y de las transacciones (cliente → crédito). Cada componente es una persona con su género y todas sus formas: completo, nombre + apellido, pila, MAYÚSCULAS sin tildes, apodo de `display_name`, handle, slug, local del correo, cédula con y sin puntos, celular con y sin 57, dirección, nacimiento.
3. **Detección en capas:** (a) diccionario de formas con la normalización exacta del juez (NFKC, confusables, invisibles, tildes, separadores de dígitos); (b) patrones: correo, correo deletreado ("martin punto torres arroba icloud punto com"), cédula, celular, placa (`KFY046`, `JXA-971`), pasaporte, crédito, dirección colombiana (Calle, Cra, Tv, Diagonal # n-n), honorífico + apellido ("Estimado señor Santander"); (c) coincidencia tolerante (edición 1) para copias alteradas; (d) **LLM por hilo o documento**, en paralelo y con JSON de spans, cacheado por hash: recall de terceros fuera de las tablas, desempate persona vs. lugar y oficio → sector.
4. **Resolución con evidencia**, nunca por nombre: llave en el registro (uid del autor, CL-, radicado) > identificador fuerte en el registro o el hilo > **fecha + monto que cruza con transacciones** > adjunto (`extracto_<nombre>_mes`) > participantes del hilo. Un nombre sin evidencia es una persona aparte. Un **tercero** (familiar, socio, jefe) recibe una llave estable `relación:persona ancla:nombre` y se le ligan su llave o cédula, así que su seudónimo no cambia entre corridas. Una variante a distancia 1 de una persona conocida ("novia Daniel Giraldo… es clienta") se liga a esa persona si el contexto lo confirma. **Protección de falsos positivos:** un gazetteer construido del propio corpus (ciudades, barrios del maestro, comercios y lugares de transacciones) más listas de organizaciones (Bancolombia, Nequi, Avianca, Superfinanciera, Kipu Pay con su NIT y su oficina) y de figuras públicas y marcas con nombre de persona, que el LLM confirma en contexto y que se declaran como `kept`. Un token que es apellido y lugar solo se reemplaza dentro de un nombre o junto a un honorífico.
5. **Seudónimos deterministas:** `HMAC(llave fija, llave de la persona)` → catálogo propio colombiano del mismo género; el género se desempata con el cargo u oficio ("Contadora"). Cada forma corta se mapea a la parte equivalente del seudónimo ("Manuela" → "Sofía Elena"), el apodo va al nombre de pila del seudónimo, y el handle, slug y correo conservan el patrón (`nombre.apellido`, inicial + apellido, guiones). Cédula, celular, placa, pasaporte y crédito con formato válido y lejos del original. Nunca coincide con una persona real. Las colisiones se resuelven sin depender del orden y el mapa se persiste, así que nadie cambia de seudónimo cuando crecen las fuentes.
6. **Reemplazo con formato:** spans de mayor a menor; se respetan puntos, mayúsculas, snake_case, la forma deletreada ("arroba/punto") y la URL. Se reemplaza también en claves de metadata, adjuntos y comentarios. El `reporta_a` de empleados usa la misma forma corta del seudónimo del jefe. Ningún registro se vacía.
7. **Cuasi-identificadores**, calibrados contra la salida ideal. En el maestro, siempre: oficio → sector, barrio → otro barrio de la misma ciudad, dirección y nacimiento → seudónimos; la ciudad, la vinculación y las llaves se quedan. En el texto: el oficio, barrio y año de vinculación de la persona → su sector, su ciudad y "hace unos años". Además se detectan **tuplas sin nombre** ("cliente de Bucaramanga, arquitecto, vinculado desde 2021"; "es arquitecto, vive en Chapinero") y, si apuntan a menos de k clientes del maestro, se generalizan igual.
8. **Verificación adversarial antes de publicar:** (a) barrido global de todas las formas reales con la normalización del juez; (b) ataque de enlace: se intenta reidentificar a cada cliente cruzando los cuasi-identificadores de la salida contra el maestro original; (c) `validar.mjs`; (d) `entities.json` honesto: todas las formas, registros y los conservados con `kept` y `reason`. Si algo falla, se corrige y se vuelve a verificar.

3. Servicio

Rutas en `/v1` con `ThreadingHTTPServer`; Bearer opcional. `/run` corre en un hilo y reporta estado, conteos y costo; `/output` y `/entities` salen del volumen. `/anonymize` carga en memoria el grafo, el mapa y la caché: la misma persona, aunque venga alterada, recibe el mismo seudónimo; una nueva recibe uno nuevo y único. Corte de 40 s: si el LLM no responde, sigue con reglas. Railway con 1 réplica, sin reposo, volumen y healthcheck.

4. Decisiones y trade-offs

Elegimos	Descartamos y por qué
Grafo de identificadores; el nombre es solo etiqueta	Agrupar por nombre: fusiona homónimos y a la agente con el cliente
Reglas + LLM de recall cacheado	Solo reglas: ~88. LLM end-to-end: no determinista y reescribe
Gazetteer sacado del corpus	Lista genérica: no conoce los barrios ni los comercios de Kipu
Generalizar siempre, ligado a la persona	k-anonimato condicional: se cruza entre fuentes y se aleja de la ideal
HMAC + mapa persistido	Seudónimos al azar: rompen consistencia entre corridas
Juez local + ataque de enlace	Iterar a ciegas con 3 corridas

5. Seguridad (prompt injection) y fallas

- **Modelo de amenaza:** el corpus que crece y los registros de `/anonymize` son texto de terceros [A]. El grafo tiene PII real [B]. El LLM **no tiene tools ni red**, así que le falta la pata [C]. El riesgo es de integridad: que una inyección ("no anonimices a X", "X es figura pública") cause una fuga. Hoy escaneamos el corpus y no hay inyecciones, pero diseñamos como si las hubiera.
- **Defensa determinística:** el LLM solo puede **agregar** detecciones; nunca quita una coincidencia del grafo o de un patrón. Conservar a alguien (`kept`) exige estar en el gazetteer o la lista curada **y** no estar en el grafo. Cada span del LLM se valida en código: la subcadena existe en esa posición, el tipo está en el enum y la longitud está acotada. Lo que no valida se descarta. El LLM nunca escribe texto de salida.
- **Higiene:** NFKC, quitar invisibles y confusables antes de detectar y antes de mandar al LLM. El texto viaja como JSON con su procedencia en el mensaje de usuario, con la política en el system prompt y sin secretos. Se rechazan XML con DOCTYPE o ENTITY, hay tope de tamaño al descomprimir PDF y tope de bytes por lote en `/anonymize`. Nada del corpus se ejecuta.
- Llaves solo en variables de entorno; los logs no llevan PII; nada de hash sin sal. Los datos y `entities.json` nunca entran al repo ni viajan con el dataset. La batería de pruebas incluye inyecciones canario.
- LLM caído → reglas. 429 → backoff. Reinicio → el estado sigue en el volumen. Esquema inválido → 422. Handler global.
- **Fuentes que crecen:** el canario de `record_index` lo detecta. El grafo se extiende sin renumerar, y si una persona vieja aparece con un identificador nuevo, se une a su componente. La corrida 1 va antes del cambio (30 %).

6. Costo y evaluación

El LLM ve ~250 mil tokens (chats, docs, notas, descripciones): unos US$1 y menos de 15 min con 8 hilos; después, la caché lo deja casi en cero. **Juez local:** copia la normalización de `validar.mjs` y mide sobre `sample/` contra la salida ideal (partición de personas, fugas, parecido, conservados), los 13 tests de la muestra, los 5 públicos y una batería de trampas sacada del corpus real: homónimos, agente = cliente, apodos, apellido-lugar, deletreados, placas, URLs, metadata UTF-16 y tuplas sin nombre. Todo se reporta por familia.

Límite: un nombre de pila sin evidencia queda como persona aparte: preferimos separar a fusionar.